[승코의 차근차근 Flutter] 06. Provider(1)

🧱 0. Provider란?

상태 관리의 핵심 철학

- 앱에는 다양한 데이터(상태)가 존재해: 예) 로그인 정보, 메뉴 목록, 장바구니, 알림 상태 등
- 이런 데이터는 여러 화면과 위젯에서 **공유**되어야 해.
- 단순한 전역 변수는 유지보수가 어렵고 비효율적.
- 그래서 **Provider**는 "**공급자(Provider)**"로서 앱 전체에 상태를 **주입하고 추적**해주는 역할을 함.

---

1. Provider – 가장 기본적인 공급자

✅ 특징

- 상태가 **변하지 않는 값**을 앱 전역에서 공유할 때 사용

- **읽기 전용** 데이터 (ex. 환경설정, 테마, 초기 구성 정보)

✅ 구조

```
Provider<T>(
  create: (context) => YourInstance(),
  child: YourWidgetTree(),
)
```

✅ 예시: 카페 정보 주입

```
class CafeInfo {
  final String name;
  final String openTime;
  CafeInfo({required this.name, required this.openTime});
}
```

```
void main() {
  runApp(
    Provider(
      create: (context) => CafeInfo(name: 'Grok 카페', openTime: '08:00 AM'),
      child: MyApp(),
    ),
  );
}
```

사용:

```
final cafeInfo = Provider.of<CafeInfo>(context, listen: false);
Text('카페 이름: ${cafeInfo.name}');
```

💡 팁

- `listen: false`를 사용하면 상태 변화를 감지하지 않고 값만 읽음.

- 자주 바뀌지 않는 설정이나 기본 정보는 `Provider`로 제공하자.

---

2. ChangeNotifierProvider – 실시간 상태 반영

✅ 특징

- 상태가 바뀌면 `notifyListeners()` 호출로 UI 자동 갱신
- `ChangeNotifier`를 상속받은 클래스를 통해 사용

✅ 예시: 선택된 음료 + 재고 관리

```
class CafeModel with ChangeNotifier {
  List<Map<String, dynamic>> _menu = [
    {'name': '아메리카노', 'stock': 10},
    {'name': '라떼', 'stock': 5},
  ];
  String _selectedDrink = '아메리카노';

  List<Map<String, dynamic>> get menu => _menu;
  String get selectedDrink => _selectedDrink;

  void selectDrink(String drink) {
    _selectedDrink = drink;
    notifyListeners();
  }

  void orderDrink(String drink) {
    final index = _menu.indexWhere((item) => item['name'] == drink);
    if (_menu[index]['stock'] > 0) {
      _menu[index]['stock']--;
      notifyListeners();
    }
  }
}
```

```
ChangeNotifierProvider(
  create: (context) => CafeModel(),
  child: MyApp(),
);
```

💡 팁

- UI에서 `Consumer`, `Provider.of`, `context.watch` 등으로 값 접근 가능

---

3. MultiProvider – 여러 Provider를 한 번에

✅ 특징

- 중첩된 Provider를 **한 번에 관리**할 수 있음
- 앱 규모가 커질수록 **필수 구조**

```
MultiProvider(
  providers: [
    ChangeNotifierProvider(create: (_) => CafeModel()),
    Provider(create: (_) => CustomerModel(name: 'VIP')),
  ],
  child: MyApp(),
);
```

💡 팁

- `MultiProvider`는 `providers` 리스트 안에 다양한 Provider를 선언함.
- 어떤 순서로 선언했는지가 중요할 수 있으니 의존 관계에 주의!

---

4. Consumer – 부분 위젯만 업데이트

✅ 특징

- `ChangeNotifier`에 등록된 모델 중 일부 위젯만 rebuild

- 전체가 아닌 **해당 위젯만 리렌더링 → 성능 향상**

```
Consumer<CafeModel>(
  builder: (context, cafe, child) {
    return Text('선택된 음료: ${cafe.selectedDrink}');
  },
)
```

💡 팁

- `child` 매개변수에 **변하지 않는 위젯**을 전달하면 더 최적화됨
- `Consumer`는 내부적으로 `ChangeNotifier`를 `watch`하고 있음

---

5. Selector – 값 변경 감지 최적화

✅ 특징

- 특정 값만 감지해서 UI 갱신
- 값이 바뀌지 않으면 rebuild 방지 (성능 향상)

```
Selector<CafeModel, String>(
  selector: (context, cafe) => cafe.selectedDrink,
  builder: (context, selected, child) {
    return Text('선택: $selected');
  },
)
```

💡 팁

- 복잡한 모델에서 모든 걸 감지하는 대신, 필요한 값만 골라내자.
- 특히 큰 리스트나 객체가 있는 경우 유용함.

---

6. ProxyProvider – 의존성 있는 Provider 연결

✅ 특징

- 하나의 Provider가 **다른 Provider**에 의존할 때 사용
- 예: 로그인된 사용자에 따라 카페 데이터 설정

```
MultiProvider(
  providers: [
    Provider(create: (_) => CustomerModel(name: 'VIP 손님')),
    ProxyProvider<CustomerModel, CafeModel>(
      update: (context, customer, previousCafe) =>
        CafeModel()..setSpecialMenu(customer.name),
    ),
  ],
  child: MyApp(),
);
```

```
class CafeModel with ChangeNotifier {
  List<String> _specialMenu = [];
  void setSpecialMenu(String name) {
    if (name == 'VIP 손님') {
      _specialMenu = ['VIP 전용 메뉴'];
      notifyListeners();
    }
  }
}
```

✨ 실제 예시 조합

VIP 전용 메뉴 + 재고 관리 + 주문 기록 (복합 구조)

```
class OrderHistoryModel with ChangeNotifier {
  final List<String> _orders = [];

  List<String> get orders => _orders;
```

```
  void addOrder(String drink) {
    _orders.add(drink);
    notifyListeners();
  }
}
```

```
MultiProvider(
  providers: [
    Provider(create: (_) => CustomerModel(name: 'VIP')),
    ChangeNotifierProvider(create: (_) => CafeModel()),
    ChangeNotifierProvider(create: (_) => OrderHistoryModel()),
  ],
  child: MyApp(),
);
```

```
// 주문 처리 함수
void processOrder(BuildContext context, String drink) {
  final cafe = context.read<CafeModel>();
  final history = context.read<OrderHistoryModel>();
  cafe.orderDrink(drink);
  history.addOrder(drink);
}
```

🚀 실전 사용 꿀팁 요약

상황	Provider 종류	사용 이유
읽기 전용 데이터	`Provider`	앱 이름, 초기값, 설정
상태 변경 필요	`ChangeNotifierProvider`	재고, 선택, 로그인 상태
많은 Provider 사용	`MultiProvider`	구조 정리
부분 업데이트	`Consumer`, `Selector`	성능 향상
의존성 연결	`ProxyProvider`	유저 정보 기반 동작

✅ 정리 – Provider 확장의 핵심은?

- **데이터는 반드시 단방향 흐름으로!**
- **View ↔ Model 분리**, 코드 유지보수에 핵심
- **적절한 Provider 선택**이 앱 성능과 구조의 핵심
- **규모가 클수록 구조화**에 신경 써야 함

🎯 다음 도전 과제

1. `ProxyProvider`로 로그인 유저에 따라 권한 메뉴 보이기
2. `ChangeNotifier` 모델을 여러 개 나누고 `MultiProvider`로 결합
3. `Selector`로 성능 최적화 실습
4. `OrderHistoryModel`에 주문 시간 저장 기능 추가
5. `ProviderScope` 확장해서 특정 화면에만 Provider 주입 실험